TESTCASES: Nachos Project 1

Execution Instructions

All test cases for Tasks 1-5 have been integrated into the OS boot sequence using the *selfTest()* methods in their respective classes. These are invoked automatically by *ThreadedKernel.selfTest()*.

To reproduce the output in this document, no command-line arguments are required.

Exectrure the following commands in your terminal:

1. Compile the kernel
   cd proj1
   gmake clean
   gmake

2. Execute the tests:
   java nachos.machine.Machine

Note: The output for intentional security violation (Task 1, Test 3 and 4) uses Java t*ry-catch* blocks to catch the *AssertionFailureError*. This prevents the kernel from crashing.

Task 1: KThread.join() Tests

This test aimed to verify that the threads join successfully, bypass sleep queues if the target is already finished, and safely halt if a thread attempts to join itself or join a thread that already has been joined.

```
Starting Task 1 Tests....

Test 1: Basic Join
        Main Thread calling join on Thread B...
        Thread B is running and is about to yield
        Thread B finishing
        Main Thread joined with B


 Test 2: Finished before joining
        Thread C is running finishing NOW
```

```
        Main thread calling join on Thread C (already finished)...
        Main bypasses sleep queue


 Test 3: Multiple joins
        Main thread joining target (This is a valid join)...
        Target thread is almost done
        Main thread attempting to join again (This is invalid)...
         Success: Assertion failuer caught


 Test 4: Self joins
        main thread attempting to join itslef
         Success: Assertion failuer caught

--- Task 1 Tests Completed SUCCESS ---
```

Task 2: Condition2 Tests

Objective: Verify that condition variables properly handle critical section locks during sleep, accurately wake a single thread, accurately wake all threads, and return without crashing if wake() or wakeAll() are called on empty queues

Output:

```
 Stating Task 2 Tests....

 Test 1: Sleep and wake
         Thread A has lock. sleeping
         Main thread aquiring lock to call wake()
         Main thread called wake() and released lock
         Thread A is awake, reaquired lock, and finishing tasks

        TEST 1 SUCCESSFUL

Test 2: Multiple waiting threads...
        Thread B acquired lock, calling sleep()...
        Thread C acquired lock, calling sleep()...
        Main thread calling wake() ONCE (should only wake Thread B)...
        Thread B woke up and finishing.
        removing Thread C to proceed...
        Thread C woke up and finishing.

        TEST 2 SUCCESSFUL
Test 3: Wake all threads
        Thread D acquired lock, calling sleep()...
```

```
        Thread E acquired lock, calling sleep()...
        main thread calling wakeAll() ....
        Thread D woke up and finishing.
        Thread E woke up and finishing.

        TEST 3 SUCCESSFUL

Test 4: Empty waiting list
        Main thread calling wake() and wakeAll() with 0 sleeping threads...
        Main thread successfully returned from wake() and wakeAll() without
errors.

--- Task 2 Tests Completed SUCCESS —
```

Task 3: Alarm.waitUntil() Tests

Objective: Verifies that the Alarm class correctly evaluates thread wait times using a PriorityQueue, waking the threads with shorter sleep times before threads with longer sleep requests, regardless of order they arrive.

Output:

```
Starting Task 3 Tests....

Test 1: Basic Wait
        Sleeper calling waitUntil(500) at time 140
        Sleeper woke at time 1030 (elapsed 890)
        Success: slept >= 500 ticks and other thread ran 87 times


 Test 2: Negative wait
        Thread calling waitUntil(-10)...
        Thread returned immediately (elapsed 0)
        Success: no sleep and no error


 Test 3: Multiple threads, different times
        Thread A calling waitUntil(2000)
        Thread B calling waitUntil(100)
        Thread B woke at time 1540
        Thread A woke at time 3560
        Success: B woke before A


 Test 4: Multiple threads, same time
        Thread 0 woke (elapsed 480)
        Thread 1 woke (elapsed 480)
        Thread 2 woke (elapsed 480)
```

```
        Thread 3 woke (elapsed 480)
        Thread 4 woke (elapsed 480)
        Success: all 5 threads woke

--- Task 3 Tests Completed SUCCESS ---
```

Task 4: Communicator Tests

Objective: Verify Ada-style rendezvous synchronous message passing. Ensure threads properly sleep on empty channels, wait for handoffs, queue sequentially without overwriting current data, and execute flawlessly under heavy concurrency (race condition testing).

Output:

```
Starting Task 4 Tests....

Test 1: Basic Handoff
        Speaker sending the number 42
        Listener heard: 42
        TEST 1 SUCCESSFUL

Test 2: Speaker Arrives First
        Speaker sending 100...
        Listener heard: 100
        TEST 2 SUCCESSFUL

Test 3: Listener Arrives first
        Speaker sending 200...
        Listener heard: 200
        TEST 3 SUCCESSFUL

Test 4: Speaker Queue
        Queued Speaker 1 sending 10....
        Queued Speaker 2 sending 20....
        Queued Speaker 3 sending 30....
        Creating 2 more listeners to remove the queued threads
        Listener 1 heard: 10
        Queued Speaker 1 finished
        Listener 2 heard: 20
        Queued Speaker 2 finished
        Listener 3 heard: 30
        Queued Speaker 3 finished
        TEST 4 SUCCESSFUL

Test 5: Race Condition Test
        Total Sent Sum: 275
        Total received Sum: 275
```

```
        Test 5 SUCCESSFUL

--- Task 4 Tests Completed SUCCESS ---
```

Task 5: ReactWater Tests

Objective: Verify proper synchronization of the ReactWater condition variables. Ensure that makeWater() is only executed when exactly two Hydrogen threads and one Oxygen thread are present, and that excess threads safely block on their respective condition queues without deadlocking or consuming CPU cycles.

Output:

```
Starting ReactWater Tests....

Test 1: Basic Reaction (2H + 1O)

        water was made!

        TEST 1 SUCCESSFUL

Test 2: Hydrogen Threads Wait for Oxygen

        water was made!

        TEST 2 SUCCESSFUL

Test 3: Oxygen Waits for Two Hydrogen Threads

        water was made!

        TEST 3 SUCCESSFUL

Test 4: Excess Threads Wait for Another Reaction

        water was made!

        water was made!

        TEST 4 SUCCESSFUL

Test 5: Multiple Concurrent Reactions

        water was made!

        water was made!
```

```
        water was made!

        water was made!

        water was made!

        water was made!

        water was made!

        water was made!

        water was made!

        water was made!

        TEST 5 SUCCESSFUL

--- All ReactWater Tests Completed SUCCESS ---
```

Full Output:

```
Starting Task 1 Tests....

Test 1: Basic Join
        Main Thread calling join on Thread B...
        Thread B is running and is about to yield
        Thread B finishing
        Main Thread joined with B


 Test 2: Finished before joining
        Thread C is running finishing NOW
        Main thread calling join on Thread C (already finished)...
        Main bypasses sleep queue


 Test 3: Multiple joins
        Main thread joining target (This is a valid join)...
        Target thread is almost done
        Main thread attempting to join again (This is invalid)...
         Success: Assertion failuer caught


 Test 4: Self joins
        main thread attempting to join itslef
         Success: Assertion failuer caught

--- Task 1 Tests Completed SUCCESS ---



 Stating Task 2 Tests....

 Test 1: Sleep and wake
         Thread A has lock. sleeping
         Main thread aquiring lock to call wake()
         Main thread called wake() and released lock
         Thread A is awake, reaquired lock, and finishing tasks

        TEST 1 SUCCESSFUL

Test 2: Multiple waiting threads...
        Thread B acquired lock, calling sleep()...
        Thread C acquired lock, calling sleep()...
```

```
        Main thread calling wake() ONCE (should only wake Thread
B)...
        Thread B woke up and finishing.
        removing Thread C to proceed...
        Thread C woke up and finishing.

        TEST 2 SUCCESSFUL

 Test 3: Wake all threads
        Thread D acquired lock, calling sleep()...
        Thread E acquired lock, calling sleep()...
        main thread calling wakeAll() ....
        Thread D woke up and finishing.
        Thread E woke up and finishing.

        TEST 3 SUCCESSFUL

Test 4: Empty waiting list
        Main thread calling wake() and wakeAll() with 0 sleeping
threads...
        Main thread successfully returned from wake() and wakeAll()
without errors.

--- Task 2 Tests Completed SUCCESS ---
Starting Task 3 Tests....

Test 1: Basic Wait
        Sleeper calling waitUntil(500) at time 390
        Sleeper woke at time 1030 (elapsed 640)
        Success: slept >= 500 ticks and other thread ran 62 times


 Test 2: Negative wait
        Thread calling waitUntil(-10)...
        Thread returned immediately (elapsed 0)
        Success: no sleep and no error


 Test 3: Multiple threads, different times
        Thread A calling waitUntil(2000)
        Thread B calling waitUntil(100)
        Thread B woke at time 1540
        Thread A woke at time 3560
        Success: B woke before A
```

```
 Test 4: Multiple threads, same time
        Thread 0 woke (elapsed 480)
        Thread 1 woke (elapsed 480)
        Thread 2 woke (elapsed 480)
        Thread 3 woke (elapsed 480)
        Thread 4 woke (elapsed 480)
        Success: all 5 threads woke

--- Task 3 Tests Completed SUCCESS ---

Starting Task 4 Tests....

Test 1: Basic Handoff
        Speaker sending the number 42
        Listener heard: 42
        TEST 1 SUCCESSFUL

Test 2: Speaker Arrives First
        Speaker sending 100...
        Listener heard: 100
        TEST 2 SUCCESSFUL

Test 3: Listener Arrives first
        Speaker sending 200...
        Listener heard: 200
        TEST 3 SUCCESSFUL

Test 4: Speaker Queue
        Queued Speaker 1 sending 10....
        Queued Speaker 2 sending 20....
        Queued Speaker 3 sending 30....
        Creating 2 more listeners to remove the queued threads
        Listener 1 heard: 10
        Queued Speaker 1 finished
        Listener 2 heard: 20
        Queued Speaker 2 finished
        Listener 3 heard: 30
        Queued Speaker 3 finished
        TEST 4 SUCCESSFUL

Test 5: Race Condition Test
        Total Sent Sum: 275
        Total received Sum: 275
        Test 5 SUCCESSFUL
```

```
--- Task 4 Tests Completed SUCCESS ---

Starting ReactWater Tests....

Test 1: Basic Reaction (2H + 1O)

water was made!

        TEST 1 SUCCESSFUL

Test 2: Hydrogen Threads Wait for Oxygen

water was made!

        TEST 2 SUCCESSFUL

Test 3: Oxygen Waits for Two Hydrogen Threads

water was made!

        TEST 3 SUCCESSFUL

Test 4: Excess Threads Wait for Another Reaction

water was made!
water was made!

        TEST 4 SUCCESSFUL

Test 5: Multiple Concurrent Reactions

water was made!
water was made!
water was made!
water was made!
water was made!
water was made!
water was made!
water was made!
water was made!
water was made!
water was made!

        TEST 5 SUCCESSFUL
```

```
--- All ReactWater Tests Completed SUCCESS ---
Machine halting!
```